HAN University of Applied Sciences

Arnhem, The Netherlands

EMBEDDED SYSTEM PUZZLE BOX

Product Report

Student ..... Biaggio Gutierrez

Student Number 2157548

Student ..... Janosi Barna-Botond

Student Number 2170302

Student ..... Bauke Bergsma

Student Number 2173593

Student ..... Danyil Demchenko

Student Number 2158860

Teacher ..... Michael de Boer

Date ..... March 20, 2026

Course ..... Embedded Systems Engineering

Summary

Case study KidsBytes

The Beacon Locator Puzzle Box project from KidsBytes company was initiated to address heavily guided tours that rely on a lot of verbal explanations for a children audience in Bronkhorst, replacing them with an interactive, dynamic tour that promotes an engaging adventure of self-discovery of Bronkhorst's vision. The client, represented by Bronkhorst, a highly technical company specialized in flow measurement and control systems, had an educational reason to showcase its building.

The main goal of the project is to realize a Puzzle Box that is a minimal but practical embodiment of a smart device capable of guiding small kids through the unknown. The planned key features include autonomous location via iBeacons, a user-friendly interface, puzzles, and a reward for finishing the tour. From the outset, the project required the integration of both hardware and software, leveraging the strengths of a diverse, cross-functional team.

The team adopted a flexible development process based on priorities. Test procedures were conducted to determine the behavior of the software and hardware, allowing design validation before committing to real-world builds. A significant challenge was the limited documentation of the FDRM MXCA153 board, which caused slow initialization in both hardware and software. Additionally, the topic of a user interface for kids is something out of scope, as it demands extreme simplicity in the logic and use of the Puzzle Box.

Key technological selections included the FDRM MXCA153 board and the HM10 Bluetooth module. Development began using the VS Code IDE to accelerate prototyping and support early software development alongside hardware iterations.

Contents

1 Introduction

How do you motivate children in an environment filled with machines, complex tools, technical language, and images that are far from understandable even for adults—let alone for kids? Adding another layer in which the children audience must listen to complicated logic could become a boring nightmare for them. Therefore, the client recognizes that such struggles are unnecessary. As a consequence, a different approach must be put in place—an approach that captures the bright energy and mood of kids in order to intelligently and emotionally wrap the vision of the company Bronkhorst.

This project's main goal is to deliver a stable, educational Puzzle Box from KidsBytes that maximizes the practical teaching of Bronkhorst to society under the specific requirements of the client, providing a discovery-based approach to the company and increasing future visits. The realization will be based on the FDRM MXCA153, with the team following an iterative prototyping process and maintaining close communication with the client, who brings a commercial vision. Work will be organized in clearly defined increments—each tested and refined in consultation—and bounded by preconditions such as fixed hardware, selected components, strict cost limits, and a twelve-week project timeframe.

The report is organized to transparently guide the reader through the development journey:

- Chapter 2 summarizes the preliminary research and early decisions;
- Chapter 3 details functional requirements and describes the target system's main functions;
- Chapter 4 addresses technical design choices and underlying architecture;
- Chapter 5 documents the realization of both hardware and software;
- Chapter 6 presents the outcomes of prototype testing; finally,
- Chapter 7 discusses conclusions and recommendations, supporting future reflection and potential industrial rollout.

2 Research that defined the key drivers of the Puzzle Box

2.1 Context at a Glance

The Puzzle Box must guide children aged 8–10 through 5 rooms at Bronkhorst High-Tech (Ruurlo) via iBeacon detection, interactive puzzles, and an electronically controlled lock — all running on the FRDM-MCXA153 and HM-10 BLE module, with a companion PC application for admin and logging.

2.2 Initial Research Questions

As initial thoughts that arose in a new project, some could be answered later but others demanded immediate answers, such as the following:

a) Functionality — What is the software supposed to do?

The software runs on two platforms: the **FRDM-MCXA153 microcontroller** (embedded firmware in C) and a **PC application with a graphical user interface**. Together they must:

- Continuously scan for iBeacon BLE signals via the **HM-10 CC2541 Bluetooth 4.0 module** and determine the current location using the Major/Minor fields in iBeacon messages
- Compare the current detected location against the configured **target location** (e.g., Room 1 → Room 2 → ... → Room 5 inside Bronkhorst Hightech, Nijverheidsstraat 1A, Ruurlo)
- **Guide the user** through 5 sequential locations within the Bronkhorst building, displaying tasks, hints, or interactions at each location
- **Present and validate puzzles** at each location (e.g., sudoku, tic-tac-toe, number collection, timed tasks) — the next location is only revealed after the puzzle is solved
- **Trigger an electronic lock/unlock actuator** when the target location is reached and the puzzle is solved at the end of the last iBeacon
- **Log route data persistently** (last route traveled, time between locations, distance) in non-volatile memory so data survives power loss
- On the **PC application**: display real-time sensor/actuator status, load and visualize logged data when the box connects, and allow the **Administrator** to configure target locations via a GUI

b) External Interfaces — How does the software interact with people?

There are **two user types** with distinct interaction interfaces:

Actor	Interface	Interaction
Primary user (primary school children)	Display on the puzzle box (visualization of current location, task instructions) + output feedback: light, sound, vibration	Solves puzzles via push buttons, sensors, or input on the box itself
Administrator	PC GUI application	Sets/updates target location(s), accesses logs

Table 1: User types and their interaction interfaces

c) Performance — Speed, availability, response time, recovery time?

The project guide does not fully specify all performance numbers — these are to be determined in consultation with the client — but the following are extractable constraints:

- **Session duration**: The box must function continuously for **a maximum of 2 hours** per visit without needing a recharge or restart
- **Beacon status visibility**: The status of received beacon signals must be **clearly visible to the kids-administrator at all times** — implying the display/LED must refresh fast enough for real-time perception
- **Power resilience**: Log data must be stored **continuously** and must **survive complete power removal** (non-volatile storage required)
- **Lock response**: The electronic lock must actuate as soon as the target location is confirmed — no explicit millisecond constraint is given
- **Recovery**: Since logs are persistent, the system must resume its last known state after power is reconnected (implicit from the non-volatile log requirement)

d) Attributes — Portability, correctness, maintainability, security?

Attribute	Requirement
Portability	The box is battery-powered (power bank) and fully self-contained — must operate without any fixed infrastructure; the power bank compartment is accessible without opening the locked compartment
Correctness	Location detection must correctly match iBeacon Major/Minor fields to predefined building/room codes (e.g., Major `0x0B01` = Bronkhorst Ruurlo, Minor `0x0001` = Room 1)
Maintainability	Target locations must be **updatable** by the administrator without hardware changes — either via PC app
Security	The Administrator menu must be **protected from normal users** — regular children using the box must not be able to access configuration options
Child-friendliness	The box must be **robust and child-friendly** — designed for primary school children as the primary audience
Cost constraint	Total cost of additional parts (excluding the FRDM-MCXA153) must **not exceed €50**

Table 2: System quality attributes

e) Design Constraints — Standards, language, resources, environment?

These are hard constraints imposed by the project guide and client:

- **Required microcontroller**: FRDM-MCXA153 (NXP Cortex-M33) — no substitution allowed
- **Required BLE module**: HM-10 CC2541 Bluetooth 4.0 — mandatory for iBeacon reception
- **Programming language**: **C** for the microcontroller firmware
- **iBeacon standard**: Apple iBeacon protocol — Major (building ID) and Minor (room ID) fields are the agreed location encoding scheme; the client installs beacons, not the student team
- **Communication interface**: UART between HM-10 and microcontroller; UART/USB between FRDM-MCXA153 and PC — the message format must be researched and decided by the team
- **Non-volatile storage**: Log data must persist across power cycles
- **PC application**: Must have a graphical user interface — no console-only applications

- **Documentation standard**: APA citation standard for all research; V-model design methodology for the full project lifecycle
- **Budget**: €50 hard cap on additional components

3 Functional Requirements defined the basis of hardware

3.1 Functional Design

At simple understanding a puzzle box is mainly build for entertainment the kids on an interactive learning which waking the curiosity and make easier involve youngest minds on technical environment, dividing the floor of company Bronkhorst in specific areas where each one these will challenge mind's kids to resolve a puzzle and giving them signal-hints to reach the next area, so the developers of this puzzle box are constrained to unpredictable use of itself box which demands simplicity interface and kind instructions.

3.1.1 Functional Specifications

F1 · Location sensing — module: `ble_drv` → `location_engine`

Table 3: F1 – Location Sensing Functional Requirements

ID	Pri	Requirement
F1.1	**M**	WHEN the box is powered and the FSM is not in `FAULT`, THE SYSTEM SHALL poll the HM-10 CC2541 for iBeacon advertisements, WITHIN a scan cycle of 250 ms, MEASURED BY the `scan_task` tick observed on the log stream.
F1.2	**M**	WHEN an iBeacon advertisement is received with a Major field equal to the configured building code (e.g. `0x0B01` Ruurlo), THE SYSTEM SHALL resolve the Minor field to one of the 5 configured rooms, WITHIN one scan cycle, MEASURED BY a `LOCATE` event entering the dispatcher.
F1.3	**M**	WHEN no matching advertisement is received for 3 consecutive scan cycles (≥ 750 ms), THE SYSTEM SHALL retain the last valid room as the current location and raise a `SIGNAL_LOW` status, MEASURED BY the log entry `DISPLAY:WARNING signal_low`.
F1.4	**S**	WHEN more than one room beacon is in range, THE SYSTEM SHALL select the one whose filtered RSSI is highest over the last 2 scan cycles, MEASURED BY a unit test on `location_engine` replaying a recorded UART trace.

F2 · Puzzle & route progression — module: `route_manager` + `puzzle_engine` (FSM states `PUZZLE`, `UNLOCK_NEXT`)

Table 4: F2 – Puzzle & Route Progression Functional Requirements

ID	Pri	Requirement
F2.1	**M**	WHEN the user enters the current target room (F1.2), THE SYSTEM SHALL start the puzzle configured for that room, WITHIN 1 scan cycle, MEASURED BY the FSM transition `LOCATE → PUZZLE` in the log.
F2.2	**M**	WHEN the puzzle is solved correctly, THE SYSTEM SHALL reveal the next target room on the display, WITHIN 500 ms, MEASURED BY timestamp diff between `puzzle_solved` and `display_next_room`.
F2.3	**M**	WHEN an incorrect puzzle input is given, THE SYSTEM SHALL keep the current room active, show a retry cue, and not alter route progress, MEASURED BY the route index being unchanged in `log_model`.
F2.4	**M**	The final room's puzzle SHALL be a 5-digit code where each digit was collected in one of the previous rooms, MEASURED BY a unit test on `puzzle_engine` with collected-digit fixtures.
F4.5-W	**W**	Multiple puzzle difficulty levels per target group will not be implemented this period (Period 3 + Period 4). Re-consider after Site Acceptance Test.
F4.2-W	**W**	Room-contextual puzzles requiring external machine interaction are out of scope for this window.
F4.4-W	**W**	Time-bound countdown puzzles are out of scope for this window.

F3 · User display & feedback — module: `display_drv` + `feedback_drv` (View)

Table 5: F3 – User Display & Feedback Functional Requirements

ID	Pri	Requirement
F3.1	**M**	WHILE the FSM is in `LOCATE` or `PUZZLE`, THE SYSTEM SHALL display the current room label and the next target room label, MEASURED BY visual inspection against the test script `tour_happy_path.csv`.
F3.2	**M**	WHEN the puzzle is solved, THE SYSTEM SHALL emit a visible state change on the screen and one audible/LED cue, WITHIN 500 ms of the `puzzle_solved` event.
F3.3	**M**	WHILE any non-admin FSM state is active, THE SYSTEM SHALL display a BLE-signal indicator that refreshes at least once every 1 s, MEASURED BY frame timestamp diff in the log.
F3.4	**S**	WHILE in `LOCATE` or `PUZZLE`, THE SYSTEM SHALL display a step indicator `Room k / 5`, MEASURED BY inspection.

Table 5 *(continued)*

ID	Pri	Requirement
F3.5	**S**	WHEN the RSSI filtered value of the current target falls below -85 dBm, THE SYSTEM SHALL display a low-signal warning icon, MEASURED BY a replay of a captured low-RSSI trace.
F3.6	**C**	WHEN the FSM reaches `FINAL`, THE SYSTEM SHALL play a ≤ 3 s celebration animation before unlock, MEASURED BY frame count on the log.
F3.7	**M**	WHEN a usability session with 5 children of age 8–10 is run on week 20, $\geq 4/5$ children SHALL complete the happy path without adult prompting, MEASURED BY observer checklist. *(Replaces the unverifiable "child-friendly" wording.)*

F4 · Unlock & reward — module: `lock_drv` (FSM state `UNLOCK_NEXT` → `FINAL`)

Table 6: F4 – Unlock & Reward Functional Requirements

ID	Pri	Requirement
F4.1	**M**	WHEN the FSM enters `FINAL`, THE SYSTEM SHALL drive the lock actuator to the open position, WITHIN 500 ms, MEASURED BY the `lock_opened` log entry and a mechanical dwell test.
F4.2	**M**	WHILE the FSM is in any state other than `FINAL` or `ADMIN_UNLOCK`, THE SYSTEM SHALL hold the lock in the closed position, MEASURED BY continuous PWM duty-cycle inspection.
F4.3	**S**	WHEN the lock transitions from closed to open, THE SYSTEM SHALL emit an LED + buzzer confirmation cue, WITHIN 500 ms, MEASURED BY log timestamp diff.
F4.4	**M**	WHEN the administrator issues `admin_unlock` via USB-CDC, THE SYSTEM SHALL drive the lock open for manual recovery, MEASURED BY round-trip time ≤ 1 s between command and `lock_opened` event.

F5 · Admin mode — module: `usb_drv` + `config_store` (+ `V_PC`)

Table 7: F5 – Admin Mode Functional Requirements

ID	Pri	Requirement
F5.1	**M**	WHEN the administrator connects over USB-CDC and sends the correct shared secret, THE SYSTEM SHALL enter `ADMIN` state and disable normal-user inputs, MEASURED BY the log entry `FSM:ADMIN_ENTER`.

Table 7 *(continued)*

ID	Pri	Requirement
F5.2	**M**	WHILE in `ADMIN`, THE SYSTEM SHALL accept create-read-update-delete operations on the target-room list (5 rooms max) and the room-to-puzzle-answer map, MEASURED BY a round-trip test from the PC app.
F5.3	**M**	WHILE not in `ADMIN`, THE SYSTEM SHALL reject all admin commands, MEASURED BY the log entry `SYS:ERROR admin_denied`.
F5.4	**C**	WHILE in `ADMIN`, THE SYSTEM SHALL be able to return a dump of the current configuration for pre-tour verification.

F6 · Companion PC application — module: `V_PC` + `usb_drv`

Table 8: F6 – Companion PC Application Functional Requirements

ID	Pri	Requirement
F6.1	**M**	WHEN the puzzle box is connected, THE PC APP SHALL open a serial session within 2 s and display a connection indicator, MEASURED BY Windows/Linux wall-clock.
F6.2	**M**	THE PC APP SHALL allow configuring the 5 target rooms (Major/Minor) and the per-room puzzle answer, persisting the change to the box within 1 s, MEASURED BY an `ADMIN:CONFIG_OK` acknowledgment.
F6.3	**M**	WHEN a session log is requested, THE PC APP SHALL download it and show: ordered route, time-per-room, total session time, MEASURED BY comparing against the on-box `log_model` ground truth.
F6.4	**S**	THE PC APP SHALL export the session log to a single CSV file with a fixed header. *(One format, per §4.3.7 Modifiable.)*

F7 · Power & persistence — module: `PWR` + `nvm_drv` (cross-cutting)

Table 9: F7 – Power & Persistence Functional Requirements

ID	Pri	Requirement
F7.1	**M**	THE POWER BANK SHALL sit in a compartment accessible without opening the electronically locked chamber, MEASURED BY mechanical inspection.
F7.2	**M**	WHEN the FSM transitions between states, THE SYSTEM SHALL persist `{room_id, timestamp, outcome}` to non-volatile memory, WITHIN 50 ms, MEASURED BY a power-cut test that reboots and recovers the log.

Table 9 *(continued)*

ID	Pri	Requirement
F7.3	**M**	WHEN power is restored after a cut, THE SYSTEM SHALL resume in `IDLE` with the last persisted session still readable by the PC app, MEASURED BY log round-trip.
F7.4	**S**	WHEN the battery voltage drops below the threshold that guarantees 15 minutes of autonomy, THE SYSTEM SHALL display a low-battery warning icon, MEASURED BY bench test with a programmable supply.
F7.5-W	**W**	Remaining-autonomy estimation in minutes is out of scope this window.

F8 · Logging (cross-cutting) — module: `LOG` + `usb_drv`

Table 10: F8 – Logging Functional Requirements

ID	Pri	Requirement
F8.1	**M**	The firmware SHALL use the `log.h` API (`Log`, `LogWithNum`, `LogSetOutputLevel`, `LogGlobalOn/Off`) exclusively; no `printf` / `iostream` calls SHALL appear in any `.c` or `.cpp` file, MEASURED BY a CI grep on every commit.
F8.2	**M**	The logger SHALL support the six levels `NONE/INFO/DEBUG/WARNING/ERROR/CRITICAL` and the six subsystems `SYS/COMMS/DISPLAY/SENSOR/FSM/POWER`, MEASURED BY a unit test that sets and reads back each level.
F8.3	**M**	WHEN `LogGlobalOff()` is called, subsequent log calls SHALL add $\leq 1\,\mu$s each on the FRDM-MCXA153, MEASURED BY an oscilloscope on a GPIO toggled around the call.
F8.4	**S**	The log stream over USB-CDC SHALL use a newline-terminated ASCII framing to enable `tail`-style inspection in a terminal.

3.1.2 Traceability Matrix

Table 11: Traceability matrix — FR ↔ architecture module ↔ FSM state

FR	Module in diagram	FSM state reached
F1.1–F1.4	`ble_drv`, `location_engine`	`LOCATE`
F2.1–F2.4	`puzzle_engine`, `route_manager`	`PUZZLE → UNLOCK_NEXT`
F3.1–F3.7	`display_drv`, `feedback_drv`, `V_ICONS`, `V_FB`	all user-facing
F4.1–F4.4	`lock_drv`, `MECH`	`FINAL`, `ADMIN_UNLOCK`
F5.1–F5.4	`usb_drv`, `config_store`, `V_PC`	`ADMIN`

Table 11 *(continued)*

FR	Module in diagram	FSM state reached
F6.1–F6.4	`V_PC` ↔ `usb_drv`	(host side)
F7.1–F7.4	`PWR`, `nvm_drv`	cross-cutting
F8.1–F8.4	`LOG`	cross-cutting

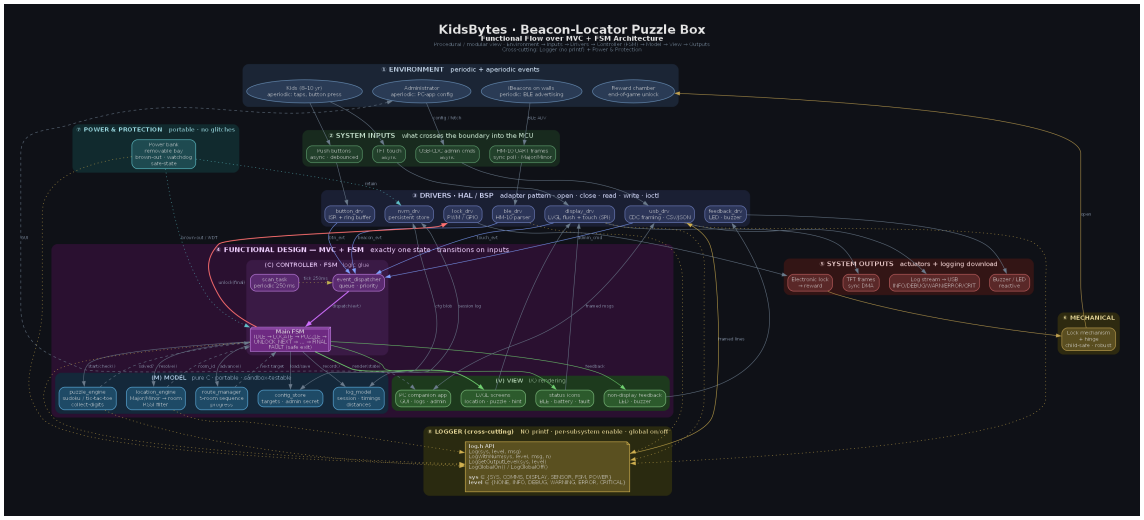

Figure 1: `component7Out4Decoder` Waveform Simulation — IT-D01 through IT-D05